PrismMathQA: Agentic Math Tutoring with AVSD

Group 28
NLP Capstone Project

Abstract

PrismMathQA is an agentic mathematics tutoring framework that combines retrieval-augmented generation, student memory, and symbolic tool use to make LLM-based tutoring more grounded, adaptive, and reliable. The system uses a math-specialized corpus built from MetaMathQA, retrieves semantically similar examples with a Qwen3-Embedding-0.6B embedder and Chroma vector search, prompts a Qwen3-4B reasoning model finetuned with Adaptive-View Self-Distillation (AVSD) using retrieved examples and learner context, and delegates exact arithmetic or algebraic verification to a SymPy-backed tool when needed. This report documents the framework at a system level and positions AVSD as the finetuning method for training the reasoning model behind PrismMathQA. AVSD treats full solutions, partial rationales, final answers, and concise hints as four privileged teacher views, then reconstructs token-level supervision by separating cross-view consensus from gated view-specific residual information. The report focuses on mathematical reasoning datasets and reports results on reasoning benchmarks including GSM8K, AIME24, AIME25, and HMMT25.

1 Introduction

Mathematics tutoring is a demanding setting for language-model systems because useful answers must be correct, pedagogically clear, and adapted to the learner's current state. A model that only emits a final answer is insufficient: students need intermediate reasoning, explanations at the right level of detail, continuity across turns, and a way to avoid small symbolic or arithmetic errors that can invalidate an otherwise useful solution. At the same time, a tutoring demo must be inspectable. When the system uses retrieved examples, a calculator, or stored student context, those components should be visible enough for a developer or evaluator to understand why the tutor answered as it did.

PrismMathQA addresses this setting with a compact agentic architecture. The framework combines a MetaMathQA-based retrieval corpus, a Qwen3-Embedding-0.6B semantic embedder, Chroma vector search, persistent learner memory, an explicit symbolic verification tool, and a Qwen3-4B reasoning model finetuned through self-distillation with AVSD. The design goal is not only to produce an answer, but to expose the right support around the answer: similar worked examples for grounding, learner context for personalization, and a symbolic verifier for computations where exactness matters.

The scope of this report is mathematical reasoning. PrismMathQA uses MetaMathQA for both retrieval corpus construction and AVSD finetuning data, and uses GSM8K, AIME24, AIME25, and HMMT25 for checkpoint reporting.

The central research question for the broader framework is how to make the model behind the tutor better at step-by-step mathematical reasoning. Standard supervised fine-tuning learns from static reasoning traces, but those traces may not match the model's own rollout distribution. Reinforcement learning with verifiable rewards, including GRPO-style training, can reward correct final answers but often supplies sparse outcome-level feedback [Shao et al., 2024, DeepSeek-AI, 2025]. Distillation provides dense distribution-level supervision [Hinton et al., 2015]; on-policy self-distillation is attractive because it trains on the model's own trajectories while using a teacher distribution to

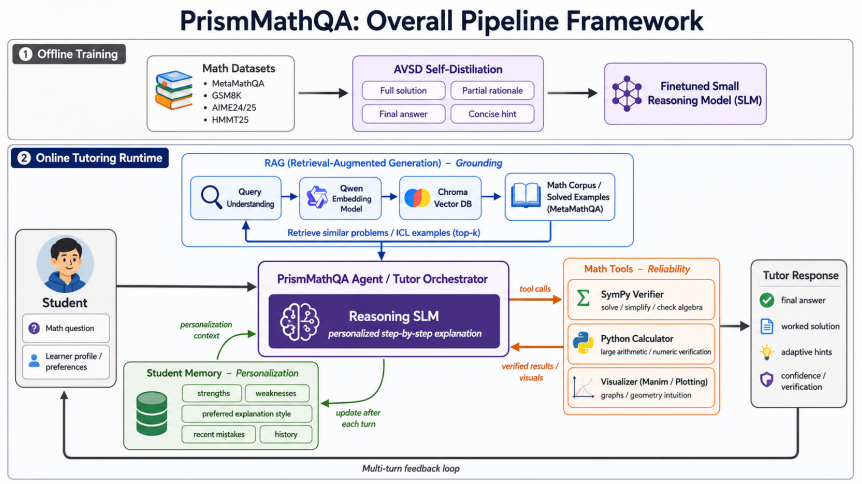


Figure 1: Overall PrismMathQA pipeline: AVSD trains the reasoning model offline, while RAG, memory, and math tools support the online tutor.

provide dense token-level learning signals [Agarwal et al., 2024, Zhao et al., 2026]. However, a single privileged teacher context, such as a full reference solution, can be too rigid or can leak information that the student will not observe at test time.

We therefore present Adaptive-View Self-Distillation (AVSD) [Nguyen et al., 2026] as the training method for the PrismMathQA reasoning model. AVSD uses four privileged views of the same training example: a full solution, a partial rationale, a final answer, and a concise hint. It first extracts a stable consensus direction across teacher views and then gates the residual support that is specific to one or a few views. This produces a reconstructed target distribution for reverse-KL on-policy self-distillation. The resulting training signal is dense like distillation, on-policy like RL-style rollout training, and more robust than committing to one privileged context.

This report makes two main contributions. First, it documents the PrismMathQA framework, showing how RAG, learner memory, and symbolic tool calling work together in an agentic math tutor. Second, it presents AVSD as the training method for the reasoning model, deriving the self-distillation objective and analyzing its benchmark behavior on Qwen3-4B reasoning checkpoints.

2 Framework Overview

PrismMathQA is designed as an agentic tutoring framework rather than a single prompt template. The central model is a Qwen3-4B reasoning model finetuned through self-distillation with AVSD, and it produces the final student-facing explanation. Around it, the framework adds three forms of support: retrieval-augmented generation for mathematical grounding, learner memory for personalization across turns, and symbolic tool calling for exact computation. These components solve different problems and are intentionally complementary. RAG gives the model relevant examples, memory gives it student context, and tool use gives it reliable computation.

Figure 1 summarizes PrismMathQA as two connected stages. In the offline stage, math training examples from a MetaMathQA subset are transformed into four AVSD supervision views: full solution, partial rationale, final answer, and concise hint. These views are used to finetune the Qwen3-4B reasoning model before deployment. In the online stage, the tutor handles each student question through an agentic loop: RAG retrieves relevant MetaMathQA examples with Qwen3-Embedding-0.6B and Chroma, learner memory supplies personalization context, and the SymPy verifier checks exact symbolic steps when needed. The final response is therefore produced by the finetuned reasoning model while being grounded, personalized, and verified by supporting modules.

2.1 Agentic Components

In ordinary question answering, the model receives only the current prompt and must rely on its parametric knowledge. That setup is fragile for tutoring. A student may ask a problem whose solution depends on a contest-math pattern; the model may know the pattern but fail to recall the right derivation. The same student may then ask follow-up questions that require continuity. Finally, even if the reasoning plan is correct, a small arithmetic error can damage trust. PrismMathQA addresses these issues by separating the tutoring task into agentic components.

The first component is the *reasoning model*. It is responsible for language, pedagogy, and mathematical explanation. It decides how much detail to show, how to adapt to the learner, and how to turn retrieved examples or tool outputs into a coherent response. The second component is the *retrieval module*, which searches a math corpus for semantically similar solved examples. The third component is *memory*, which keeps track of learner preferences and recent interaction history. The fourth component is a *symbolic verification tool*, which checks exact algebra or arithmetic when the model should not rely on free-form generation.

These components are not independent add-ons. They form a division of labor. The reasoning model is strongest at natural language explanation and at connecting mathematical steps into a coherent narrative. The retriever is strongest at finding analogous problems and solution patterns. Memory is strongest at carrying learner-specific context across turns. The symbolic tool is strongest at deterministic computation. PrismMathQA uses the LLM as the coordinator, but it does not require the LLM to solve every subproblem alone. This is the main reason the framework is agentic: the answer is produced through controlled interaction between specialized sources of support.

2.1.1 Retrieval-Augmented Generation

RAG is important because mathematical reasoning is highly pattern-sensitive [Lewis et al., 2020]. A problem about a parameterized curve, a geometry perimeter, or a polynomial factorization often resembles earlier worked examples. If the model receives relevant examples in context, it can align its explanation style and decomposition strategy with the target domain. This does not mean that retrieved examples replace reasoning. In PrismMathQA, retrieved examples act as demonstrations: they help the LLM choose a useful solution pattern, but the final answer must still solve the student's exact question.

The retrieval corpus is based on MetaMathQA, a large collection of mathematical question-answer pairs and worked solutions. PrismMathQA embeds questions with Qwen3-Embedding-0.6B and stores the resulting vectors in Chroma. At inference time, the student's question is embedded with the same embedder, nearest examples are retrieved, and the reasoning model receives them as in-context demonstrations. This design gives the tutor domain-specific mathematical context without requiring the base LLM to memorize every example.

Using Qwen3-Embedding-0.6B as the embedder is useful because the retriever should understand mathematical semantics rather than only surface-level word overlap. Two problems can share a solution strategy even when their wording is different. For example, a quadratic vertex problem and a completing-the-square example may not share many tokens, but they are close in mathematical structure. Dense embeddings help retrieve by conceptual similarity, while Chroma provides a practical vector-search backend for storing and querying those representations.

In PrismMathQA, retrieval is especially valuable for medium and hard problems. Easy arithmetic questions may not need retrieved context, but contest-style problems often require choosing the right representation. RAG can expose whether a problem should be approached by parameter elimination, factoring, coordinate distance formulas, modular arithmetic, or completing the square. This helps the model choose a starting point before it begins to write the explanation.

Table 1 illustrates why MetaMathQA is appropriate for PrismMathQA. The corpus is not just a list of short arithmetic questions. It contains diverse mathematical structures: symbolic equations, coordinate geometry, function analysis, and competition-style derivations. This diversity matters because the retriever can supply examples at different levels of abstraction. For an easy quadratic problem, a simple worked example may be enough. For a harder parametric curve problem, the system needs examples that demonstrate how to introduce variables, eliminate parameters, and present a final symbolic form.

Table 1: Representative MetaMathQA-style examples and why they are useful as a PrismMathQA retrieval corpus.

Question type	Concrete sample question	Mathematical skill	Why it supports RAG tutoring
Algebraic manipulation	Solve for x: $987654321x - 123456789 = 555555555$. Give the answer as a reduced fraction.	Linear equations, exact arithmetic, fraction reduction.	Retrieves examples that show clean symbolic transformations and exact intermediate steps.
Geometry and coordinate reasoning	A hexagon has consecutive vertices $(0,1)$, $(1,2)$, $(2,2)$, $(2,1)$, $(3,1)$, $(2,0)$, and $(0,1)$. If the perimeter is $a+b\sqrt{2}+c\sqrt{5}$, find $a+b+c$.	Distance formula, radical simplification, geometric modeling.	Retrieves demonstrations that translate diagrams or coordinates into algebraic quantities.
Functions and graph interpretation	Given $f(x) = x^2 - 4x + 1$, find the vertex, axis of symmetry, and minimum value.	Completing the square, quadratic interpretation.	Supplies high-school level explanations that connect algebraic manipulation to graph meaning.
Advanced contest reasoning	A curve is $(x, y) = (2\cos t - \sin t, 64\sin t)$ and can be written as $ax^2 + bxy + cy^2 = 1$. Find (a, b, c).	Parameter elimination, trigonometric substitution, implicit forms.	Gives harder multi-step patterns where in-context solution structure is especially valuable.

MetaMathQA is also suitable because its examples include both questions and solution text. A retrieval corpus for tutoring should not only retrieve the answer; it should retrieve a style of reasoning. The model can observe how a solution is broken into steps, how notation is introduced, and how the final result is justified. This makes the corpus useful as an in-context teaching source, not merely as an answer lookup table.

The corpus is also compatible with the AVSD data format. The same mathematical artifacts that make MetaMathQA useful for RAG also make a MetaMathQA subset suitable for multi-context finetuning. A worked solution can be transformed into a full-solution view, a truncated partial-solution view, a final-answer view, or a concise hint. This means the runtime retrieval corpus and the training-time privileged views are aligned in structure: both are built from solved MetaMathQA examples, but they are used differently. At inference time, examples are retrieved as demonstrations. During finetuning, examples provide teacher views for token-level supervision.

2.1.2 Memory

Memory is important because tutoring is a longitudinal interaction. A model that forgets the learner after every turn cannot adapt its explanation style or build on prior clarification. PrismMathQA uses memory to represent two kinds of context. The first is a learner profile: grade level, preferred explanation style, strengths, and weak areas. The second is recent conversation history: previous questions, model answers, and tool or retrieval traces.

At a high level, memory solves three problems. First, it supports personalization. A student who asks for step-by-step explanations should receive a different response from a student who wants concise verification. Second, it supports continuity. If the student asks "why did you divide by this term?" the tutor needs the previous answer to interpret the follow-up. Third, it supports accountability. Stored interaction summaries make it possible to audit what the tutor saw and how it responded.

Memory is particularly important for mathematics because student errors are often systematic. A learner may repeatedly confuse distribution with factoring, forget sign changes when moving terms across an equation, or apply the power rule incorrectly. Even lightweight memory can help the tutor notice that the student is asking related questions and should receive a targeted reminder. Without

memory, the model treats each query as isolated and may miss the opportunity to reinforce the same misconception across examples.

PrismMathQA deliberately keeps memory lightweight. It does not require a complex student model to be useful. Even recent-turn memory can improve tutoring quality because many mistakes and follow-up questions are local. A future version can extend this into mastery tracking, misconception detection, and curriculum recommendation. The current framework treats memory as a necessary context layer that helps the reasoning model behave like a tutor rather than a stateless calculator.

There is also a pedagogical reason to keep memory interpretable. If the system adapts its explanation, the adaptation should be understandable: the tutor is using the learner's stated grade level, preferred style, or recent mistakes. An opaque memory representation may improve performance but can make the tutor harder to audit. PrismMathQA therefore treats memory as a readable context layer first, and a target for richer learner modeling second.

2.1.3 Tool Calling

Tool calling is important because mathematical fluency and exact computation are not the same ability. A reasoning LLM can explain a method correctly and still make an arithmetic error. In a tutoring system, this is costly: students may learn the wrong result or lose trust in the explanation. PrismMathQA therefore gives the model access to a symbolic verification tool based on SymPy [Meurer et al., 2017].

The tool is used when exact computation materially improves correctness: solving equations, simplifying expressions, checking nontrivial fractions, differentiating or integrating symbolic expressions, or verifying large arithmetic. The reasoning model decides whether a problem should be answered directly or whether a tool call is warranted. If the tool is used, the result is returned to the model, and the model turns that result into a student-facing explanation.

The decision to call a tool is itself part of the tutoring behavior. A good tutor should not outsource every step, because students need to see reasoning. At the same time, a good tutor should not pretend that mental arithmetic is reliable for very large numbers. PrismMathQA therefore uses tool calling selectively. Routine conceptual explanations can remain direct, while exact symbolic or numerical checks can be delegated to the verifier. This preserves the educational role of the LLM while improving factual reliability.

This separation is useful pedagogically. The tool should not replace the explanation; it should stabilize it. For example, when solving a large linear equation, SymPy can verify the exact rational solution, while the LLM explains why adding a term to both sides and reducing a fraction are valid steps. The student sees a coherent solution, but the system has reduced the risk of silent computational error.

Tool calling also creates an audit trail. If an answer depends on a computed derivative or a reduced fraction, the framework can record that the symbolic verifier was used. This matters when evaluating a tutoring system: an evaluator can distinguish between an answer that came from model-only reasoning and an answer that was supported by exact computation. The distinction is useful for debugging, for measuring tool-call appropriateness, and for deciding which failures require better prompting, better tool schemas, or better model training.

3 Fine-tuning Method: Multi-context Self-Distillation

PrismMathQA's runtime framework can improve answer grounding and verification, but it cannot by itself make the base model a better mathematical reasoner. The model still needs training signals that reward correct reasoning trajectories and discourage misleading intermediate steps. This section presents Adaptive-View Self-Distillation (AVSD) [Nguyen et al., 2026] as the finetuning method for the model used by PrismMathQA. For this report, the finetuning data is a MetaMathQA subset whose examples provide a problem statement and solution artifacts. The method is especially well matched to a math tutor because these examples naturally come with multiple contexts: a full reference solution, a partial chain of reasoning, a final answer, and a concise hint. These contexts are useful during training but are not all available to the student model at inference time.

Knowledge distillation is a standard way to transfer behavior from a teacher model or teacher distribution into a smaller or task-specialized student model [Hinton et al., 2015]. In language-

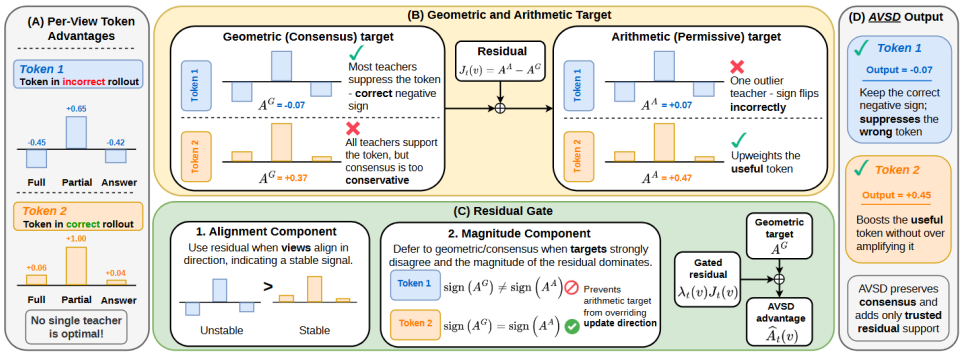


Figure 2: AVSD training pipeline used as the finetuning method for PrismMathQA. The student generates on-policy rollouts from the problem statement, while multiple privileged teacher views provide token-level supervision. AVSD separates cross-view consensus from view-specific residual signals and gates the residual before constructing the final distillation target.

model post-training, distillation is attractive because it provides richer supervision than a single binary correctness label: the student can learn from token probabilities, solution traces, or preferred reasoning styles. For mathematical reasoning, this is especially useful because many errors are local. A final answer may be wrong because of one algebraic step, one sign error, or one invalid simplification; dense distillation can in principle teach the model which intermediate tokens should become more or less likely.

However, the classical teacher-student setup is not automatically effective for reasoning. If the teacher trajectory is offline and fixed, the student may be trained on states that it will not visit at inference time. If the teacher has privileged information, such as a full solution, the student may imitate tokens that depend on information unavailable in the real prompt. If a single teacher view is chosen, the training signal can inherit that view's bias: a full solution may over-constrain the reasoning path, while a final answer may be too sparse to guide intermediate steps. AVSD addresses these limitations by keeping training on-policy and by aggregating multiple privileged views rather than trusting one teacher context.

Figure 2 summarizes the training flow. The student sees only the math problem and produces a rollout. The teacher mode evaluates the same rollout under four privileged contexts: full solution, partial solution, final answer, and concise hint. The algorithm then combines these teacher distributions into a reconstructed target that preserves stable agreement while filtering view-specific signals that may be unreliable.

3.1 From Single-view Self-Distillation to Token Advantages

Let D be a training set of examples (x, r), where x is the student-visible prompt and r is privileged training-time information. In a math setting, x can be the problem statement and r can be a reference solution or answer annotation. The student model observes only x during rollout and samples

$$y = (y_1, \ldots, y_T) \sim P_\theta^S(\cdot \mid x).$$

At position t, define the prefix

$$h_t = (x, y_{<t})$$

and the student next-token distribution

$$p_t(v) := P_\theta^S(Y_t = v \mid h_t), \qquad v \in \mathcal{V},$$

where $\mathcal{V}$ is the vocabulary.

In standard single-view self-distillation, the same model is evaluated again in a teacher mode that receives privileged context r. At the same prefix h_t, the teacher distribution is

$$q_t(v) := \text{sg}\left[P_\theta^T(Y_t = v \mid h_t, r)\right],$$

where $\mathrm{sg}[\cdot]$ denotes stop-gradient. The teacher is not a separate larger model; it is the same model under richer conditioning. The local reverse-KL distillation term is

$$\ell_t(\theta) = D_{\mathrm{KL}}(p_t \| q_t) = \sum_{v \in \mathcal{V}} p_t(v) \left(\log p_t(v) - \log q_t(v)\right).$$

Using $\nabla_\theta p_t(v) = p_t(v) \nabla_\theta \log p_t(v)$, its gradient can be written as

$$\begin{aligned}\nabla_\theta \ell_t &= \sum_{v \in \mathcal{V}} \nabla_\theta p_t(v) \left(\log p_t(v) - \log q_t(v) + 1\right) \\ &= \mathbb{E}_{v \sim p_t} \left[\left(\log p_t(v) - \log q_t(v) + 1\right) \nabla_\theta \log p_t(v)\right].\end{aligned}$$

The constant 1 is a baseline because

$$\mathbb{E}_{v \sim p_t}[\nabla_\theta \log p_t(v)] = 0.$$

Therefore,

$$\nabla_\theta \ell_t = \mathbb{E}_{v \sim p_t} \left[\left(\log p_t(v) - \log q_t(v)\right) \nabla_\theta \log p_t(v)\right].$$

Gradient descent on ℓ_t moves in the direction

$$\mathbb{E}_{v \sim p_t} \left[A_t(v) \nabla_\theta \log p_t(v)\right],$$

where

$$A_t(v) := \log q_t(v) - \log p_t(v).$$

This is the token-level distillation advantage. If $A_t(v) > 0$, the teacher assigns token v higher probability than the student, so training promotes that token. If $A_t(v) < 0$, training suppresses it. The intuition is close to policy-gradient reinforcement learning, but the reward is a dense teacher-student probability comparison at every token rather than a sparse final-answer reward.

For PrismMathQA, this is important because many wrong math answers contain both useful and harmful tokens. A sparse verifier might only say that the final answer is incorrect. The reverse-KL advantage can instead identify which generated tokens the privileged teacher would have made more or less likely at each prefix. This dense feedback is valuable for learning step-by-step reasoning.

3.2 Multi-view Privileged Contexts

Single-view self-distillation requires choosing one privileged context. In mathematics, that choice is unstable. A full reference solution gives detailed supervision but may overfit to one derivation. A partial rationale can provide useful early structure while leaving room for the student model to complete the solution. A final answer is compact and avoids forcing a chain of thought, but it gives weak local guidance. A concise hint points to the key theorem, substitution, or transformation without revealing the full derivation. AVSD treats these four contexts as a teacher family rather than selecting one.

Let the training example contain privileged information r, and let

$$\mathcal{V}(r) = \{r^{(1)}, \ldots, r^{(M)}\}$$

be M transformed views, where $r^{(m)} = T_m(r)$. For PrismMathQA math finetuning, the default setting uses four views:

$$\begin{aligned}r^{(1)} &= \text{full solution}, \quad r^{(2)} = \text{partial rationale}, \\ r^{(3)} &= \text{final answer}, \quad r^{(4)} = \text{concise hint}.\end{aligned}$$

Each view induces a teacher distribution at the same student prefix:

$$q_t^{(m)}(v) := \mathrm{sg}\left[P_\theta^T(Y_t = v \mid h_t, r^{(m)})\right], \qquad m = 1, \ldots, M.$$

The per-view token advantage is

$$\Delta_t^{(m)}(v) := \log q_t^{(m)}(v) - \log p_t(v).$$

The central problem is how to combine these advantages. Averaging all teachers too permissively can allow one view to dominate. Requiring all teachers to agree can be too conservative and can discard useful view-specific information. AVSD solves this by decomposing the multi-view signal into consensus and residual components.

Table 2 defines the four math-specific privileged contexts used in PrismMathQA. The concise-hint view is included as a first-class view because it matches tutoring behavior: it reveals a useful strategy without exposing a complete solution. In the equations below, $M = 4$ for the default PrismMathQA setup, although the same AVSD objective can be ablated with fewer views in Section 4.

Table 2: Math privileged views used by AVSD-style PrismMathQA training.

View	Teacher context	Training role
Full solution	Complete worked derivation with final answer.	Provides dense reasoning supervision and exposes the full mathematical structure of the solution.
Partial solution	Initial reasoning steps or a truncated chain of thought.	Guides the model toward a useful start without forcing every later step to match a reference trace.
Final answer	Gold answer without full derivation.	Supplies compact correctness information while allowing the model to search for its own explanation path.
Concise hint	Short mathematical hint such as the key substitution, theorem, or transformation.	Encourages the model to discover the missing derivation while still pointing it toward a productive strategy.

3.3 Geometric Consensus Target

The geometric target captures intersection support across teacher views. Define the unnormalized geometric score

$$\widetilde{q}_t^G(v) := \left(\prod_{m=1}^{M} q_t^{(m)}(v) \right)^{1/M} = \exp\left(\frac{1}{M} \sum_{m=1}^{M} \log q_t^{(m)}(v) \right),$$

and normalize it as

$$q_t^G(v) := \frac{\widetilde{q}_t^G(v)}{\sum_{u \in \mathcal{V}} \widetilde{q}_t^G(u)}.$$

The geometric mean is conservative. A token receives high support only if it is supported across views. If the full-solution teacher strongly promotes a token but the partial-rationale and answer-only teachers suppress it, the geometric score decreases. This is desirable when the token depends on details that only one privileged view reveals.

For advantage analysis, AVSD uses the unnormalized geometric score because normalization contributes only a token-independent constant. The geometric advantage is

$$\begin{aligned} A_t^G(v) &:= \log \widetilde{q}_t^G(v) - \log p_t(v) \\ &= \frac{1}{M} \sum_{m=1}^{M} \log q_t^{(m)}(v) - \log p_t(v) \\ &= \frac{1}{M} \sum_{m=1}^{M} \left(\log q_t^{(m)}(v) - \log p_t(v) \right) \\ &= \frac{1}{M} \sum_{m=1}^{M} \Delta_t^{(m)}(v). \end{aligned}$$

Thus the consensus advantage is the average signed update induced by the teacher family. If most views agree that a token should be suppressed, $A_t^G(v)$ is negative. If they agree it should be promoted, $A_t^G(v)$ is positive. For a tutor model, this consensus is the safest part of the training signal because it is less likely to depend on a single privileged annotation format.

3.4 Arithmetic Marginal Target

The arithmetic target captures union support. It is defined as

$$q_t^A(v) := \frac{1}{M} \sum_{m=1}^{M} q_t^{(m)}(v),$$

with arithmetic advantage

$$A_t^A(v) := \log q_t^A(v) - \log p_t(v).$$

This target is permissive: a token can receive high probability if any view strongly supports it. Such support can be useful. For example, an answer-only view may make a concise algebraic transition more likely, while a full-solution view may favor a detailed explanatory phrase. A good tutor should be able to learn from both signals.

The weakness of the arithmetic target is that it can preserve artifacts. A token may be strongly supported because one privileged view exposes information that the student model cannot infer from the prefix. Directly distilling from q_t^A can therefore encourage tokens whose support is not robust under the student-visible context. In educational terms, the model may learn to imitate a teacher that has already seen the solution rather than learn a reasoning step that follows from the problem.

3.5 Cross-view Residual

The gap between the arithmetic and geometric targets isolates view-specific support. AVSD defines the residual

$$J_t(v) := \log q_t^A(v) - \log \widetilde{q}_t^G(v).$$

By the arithmetic-geometric mean inequality, $q_t^A(v) \geq \widetilde{q}_t^G(v)$, so

$$J_t(v) \geq 0.$$

The arithmetic advantage can be decomposed as

$$A_t^A(v) = A_t^G(v) + J_t(v).$$

This equation is the key decomposition. $A_t^G(v)$ is the shared signal: it says what the teacher family agrees about. $J_t(v)$ is the additional support retained by the permissive target but absent from strict consensus. A small residual means the views agree. A large residual means at least one view assigns much more support than the others.

The residual is not automatically good or bad. It may contain complementary evidence that improves the model, or it may contain privileged leakage. AVSD therefore does not discard it, but also does not trust it unconditionally. The method keeps the consensus direction and gates the residual contribution.

3.6 Residual Gate

AVSD reconstructs a token-level advantage

$$\widehat{A}_t(v) := A_t^G(v) + \lambda_t(v) J_t(v), \qquad \lambda_t(v) \in [0, 1].$$

The gate $\lambda_t(v)$ has two components: an alignment component and a magnitude component.

The alignment component checks whether the per-view advantages agree in sign. It is defined as

$$C_t(v) := \frac{|A_t^G(v)|}{\frac{1}{M}\sum_{m=1}^{M} |\Delta_t^{(m)}(v)| + \epsilon} = \frac{\left|\frac{1}{M}\sum_{m=1}^{M} \Delta_t^{(m)}(v)\right|}{\frac{1}{M}\sum_{m=1}^{M} |\Delta_t^{(m)}(v)| + \epsilon}.$$

If the teacher views point in the same direction, the numerator is close to the denominator and $C_t(v)$ is high. If some views promote a token while others suppress it, the signed average can cancel and $C_t(v)$ is low. This distinguishes useful complementary support from direct conflict.

The magnitude component checks whether the residual is proportionate to the consensus:

$$R_t(v) := \frac{|A_t^G(v)|}{|A_t^G(v)| + J_t(v) + \epsilon}.$$

If the residual is much larger than the consensus, then the arithmetic target may override the direction established by the teacher family. In that case $R_t(v)$ becomes small. If the residual is moderate relative to the consensus, $R_t(v)$ allows the residual to adjust the update magnitude.

The final gate is

$$\lambda_t(v) = C_t(v) R_t(v) = C_t(v) \frac{|A_t^G(v)|}{|A_t^G(v)| + J_t(v) + \epsilon}.$$

This gate acts as an adaptive regularizer. It admits view-specific information when the views agree on direction and when the residual is not too large. It suppresses residual information when the views conflict, when consensus is weak, or when one view tries to dominate.

The boundedness property follows directly:

$$\lambda_t(v)J_t(v) = C_t(v)\frac{|A_t^G(v)|J_t(v)}{|A_t^G(v)| + J_t(v) + \epsilon} \leq |A_t^G(v)|.$$

Since $C_t(v) \leq 1$ and the denominator is at least $J_t(v)$, the gated residual cannot exceed the consensus magnitude. Therefore, if $A_t^G(v) < 0$, the reconstructed advantage cannot be flipped to a large positive value solely by a residual from one privileged view. This is the protection against privileged-view leakage: residual support can strengthen a trusted direction, but it cannot freely reverse it.

3.7 Reconstructed Target and AVSD Loss

The reconstructed target corresponding to $\widehat{A}_t$ is obtained by reweighting the geometric target with the gated residual and normalizing:

$$q_t^\star(v) = \frac{\widetilde{q}_t^G(v)\exp(\lambda_t(v)J_t(v))}{\sum_{u\in\mathcal{V}}\widetilde{q}_t^G(u)\exp(\lambda_t(u)J_t(u))}.$$

This target blends token-level signals. If $\lambda_t(v) = 0$, the token follows the geometric consensus target. If $\lambda_t(v) = 1$ for all tokens, the method recovers the arithmetic marginal target. In practice, most tokens remain anchored to consensus, and only selected tokens receive residual support.

The final training objective is on-policy reverse-KL distillation to the reconstructed target:

$$\mathcal{L}_{\mathrm{AVSD}}(\theta) = \mathbb{E}_{(x,r)\sim D}\mathbb{E}_{y\sim P_\theta^S(\cdot|x)}\left[\sum_{t=1}^{|y|} D_{\mathrm{KL}}\left(p_\theta(\cdot \mid h_t)\|\,\mathrm{sg}[q_t^\star(\cdot)]\right)\right].$$

Gradients flow through the student distribution only. The teacher views and reconstructed targets are treated as fixed supervision at the current prefix. Equivalently, the sampled-token update uses the reconstructed advantage

$$\widehat{A}_t(y_t) = A_t^G(y_t) + \lambda_t(y_t)J_t(y_t).$$

3.8 Training Procedure for PrismMathQA

In a PrismMathQA finetuning pipeline, each training iteration would proceed as follows. First, sample a math problem and its available privileged artifacts from the MetaMathQA subset. Second, generate an on-policy rollout from the current student model using only the problem statement. Third, construct the four privileged views: full solution, partial rationale, final answer, and concise hint. Fourth, evaluate the same model in teacher mode under each privileged view at the student-generated prefixes. Fifth, compute A_t^G, q_t^A, J_t, λ_t, $q_t^\star$, and the reverse-KL loss. Finally, update the model parameters by minimizing $\mathcal{L}_{\mathrm{AVSD}}$.

The computational overhead is controlled because all teacher views share the same student rollout. The expensive sequential generation path is not repeated for every view; the additional work is view-conditioned forward evaluation at existing prefixes. The AVSD paper reports a modest runtime increase from 41.5 seconds per step for single-view OPSD to 46.7 seconds per step for AVSD under the same Qwen3-4B setup [Nguyen et al., 2026]. For PrismMathQA, this makes AVSD a plausible training extension rather than an entirely different serving architecture.

The method also matches the runtime philosophy. RAG, memory, and tools each add context to the tutor, but the system controls how that context enters the answer. AVSD applies the same idea during training: privileged contexts are useful, but they must be balanced so that the student learns reasoning behavior available at inference time. The result is a model that can be deployed inside the PrismMathQA agentic loop with stronger mathematical reasoning before retrieval and tool use are applied.

Table 3: PrismMathQA benchmark table on a fixed Qwen3-4B backbone. The best result in each benchmark column is bolded, and the second-best result is underlined.

Qwen3-4B checkpoint		Reported score			
Method	Training signal	GSM8K	AIME24	AIME25	HMMT25
Base	None	94.95	72.5	64.16	39.16
SFT	Static traces	89.91	56.3	49.83	29.6
GRPO	Verifiable rewards	95.81	73.14	65.70	41.82
OPSD	Single privileged view	94.98	73.31	65.46	43.31
RLSD	RLVR + self-distillation	**96.89**	**77.82**	<u>67.98</u>	<u>43.59</u>
AVSD	Multi-context views	<u>96.77</u>	<u>76.20</u>	**68.29**	**44.25**

4 Experiments

This section defines the evaluation layout for PrismMathQA. The goal is to prepare a consistent reporting format for future model checkpoints trained with AVSD. As stated in the introduction, the report is restricted to mathematical reasoning datasets, so the benchmark suite contains GSM8K, AIME24, AIME25, and HMMT25.

4.1 Evaluation Setting

The evaluation suite contains four math benchmarks:

- **GSM8K**: grade-school arithmetic word problems that test multi-step numerical reasoning [Cobbe et al., 2021].
- **AIME24**: American Invitational Mathematics Examination 2024, representing competition-style algebra, geometry, number theory, and combinatorics [Zhang and Math-AI, 2024].
- **AIME25**: American Invitational Mathematics Examination 2025, used as a newer competition benchmark [Zhang and Math-AI, 2025].
- **HMMT25**: Harvard-MIT Mathematics Tournament 2025, a difficult contest-style benchmark [Balunovic et al., 2025].

The default PrismMathQA reporting metric is **Avg@4**. For each problem, the model may produce four sampled attempts, and success is measured by whether the correct answer appears among those attempts. Avg@4 is appropriate for a tutoring-oriented reasoning model because it captures whether the model can reach a valid solution under moderate sampling without relying on a large number of attempts. When values are imported directly from the AVSD paper, the table notes the paper's Avg@8 protocol.

The compared training methods are evaluated on the same Qwen3-4B backbone. The **Base** row is the unfinetuned reasoning model, while **SFT** denotes supervised fine-tuning on static mathematical traces. **GRPO** uses verifiable final-answer rewards through group relative policy optimization [Shao et al., 2024]. **RLSD** refers to RLVR with Self-Distillation, which combines verifiable rewards with self-distillation signals for fine-grained update magnitude [Yang et al., 2026]. **OPSD** is the single-view on-policy self-distillation baseline [Zhao et al., 2026], and **AVSD** extends this setting to four privileged views: full solution, partial solution, final answer, and concise hint [Nguyen et al., 2026].

4.2 Main Results

Table 3 compares finetuning methods under the same Qwen3-4B backbone, so the main variable is the training signal rather than model scale. RLSD obtains the best GSM8K and AIME24 scores, while AVSD remains second-best on both. GRPO improves over the base model on three of the four benchmarks, but the weaker SFT row suggests that static supervised traces alone are not sufficient for robust mathematical reasoning.

The strongest AVSD results appear on the harder contest-style benchmarks. AVSD is best on AIME25 and HMMT25, improving over the single-view OPSD baseline by 2.83 points on AIME25 and

Table 4: View-count ablation template for AVSD on Qwen3-4B. The table compares progressively richer privileged-view settings from a single full-solution view to the four-view AVSD setup.

AVSD view setting	GSM8K	AIME24	AIME25	HMMT25
1 view: full solution	94.98	73.31	65.46	43.31
2 views: full + partial	95.58	74.27	66.40	43.62
3 views: full + partial + answer	96.17	75.24	67.35	43.94
4 views: full + partial + answer + hint	96.77	76.20	68.29	44.25

0.94 points on HMMT25. This supports the core AVSD hypothesis: when problems require longer multi-step reasoning, aggregating several privileged teacher views can provide a more stable training target than relying on one view.

4.3 Number of Privileged Views

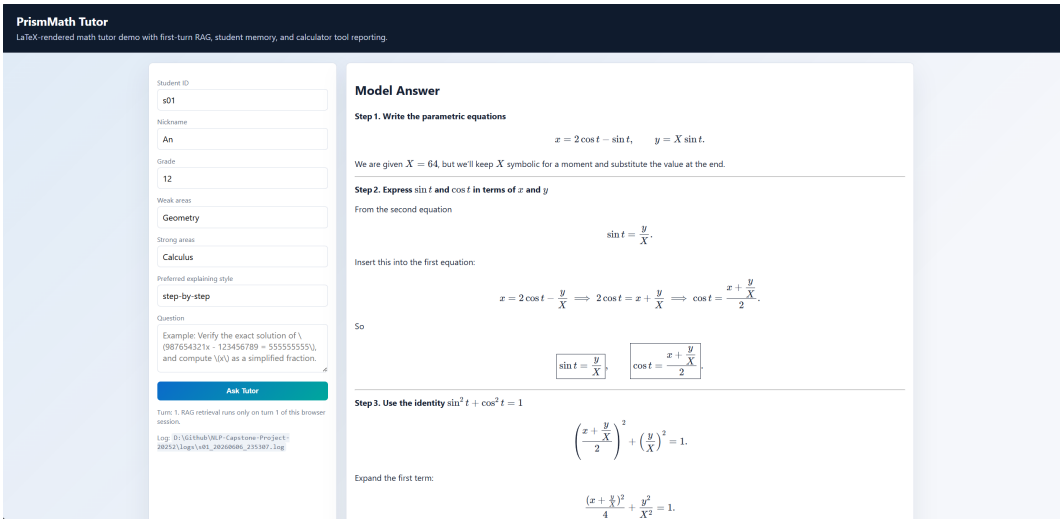
Table 4 shows a consistent pattern: performance rises as the teacher family grows from one privileged view to four. Moving from the OPSD-style single-view setting to the full AVSD setting gives gains of 1.79 on GSM8K, 2.89 on AIME24, 2.83 on AIME25, and 0.94 on HMMT25. The main insight is that view diversity is most valuable on AIME-style multi-step contest problems, where full solutions, partial rationales, final answers, and hints provide complementary supervision rather than redundant signals.

5 Demonstration

The PrismMathQA demo is evaluated through one concrete interaction rather than a broad benchmark. The initial question asks the tutor to solve the parametric-curve problem

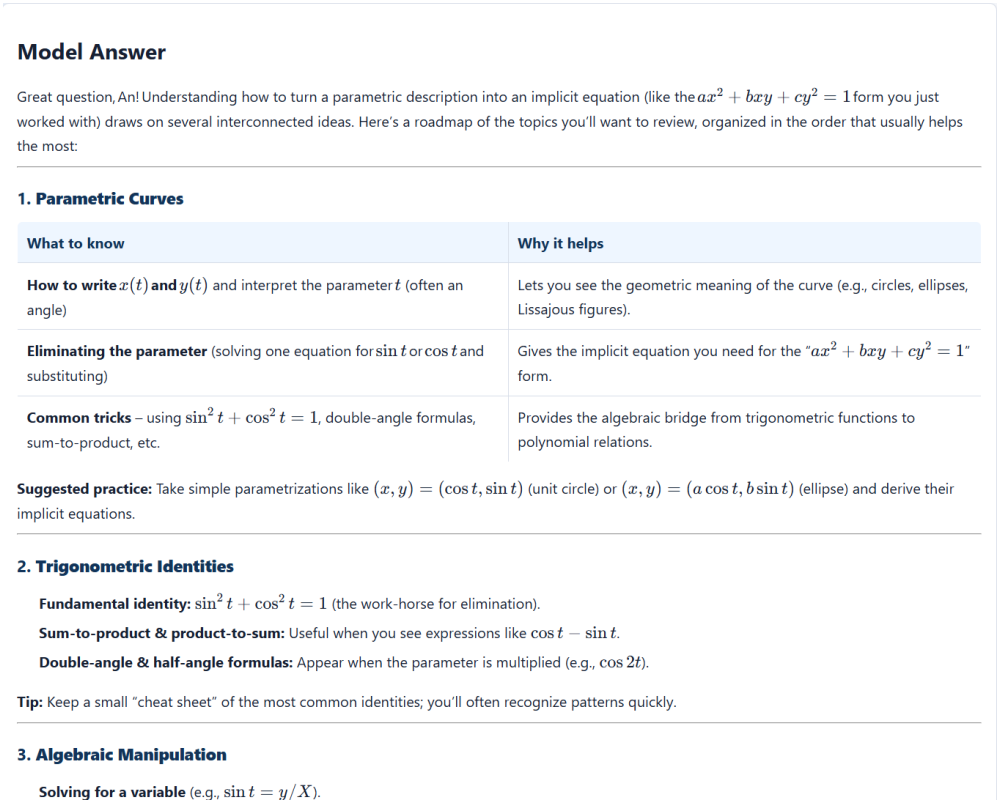
$$(x, y) = (2\cos t - \sin t, X \sin t)$$

and express the graph as $ax^2 + bxy + cy^2 = 1$ when $X = 64$. This example is useful because it requires parameter elimination, exact algebraic simplification, and a structured final triple. The first screenshot shows the first-turn web interface: the left panel records the learner profile, weak and strong areas, preferred explanation style, question box, turn metadata, and log path, while the right panel renders the model answer with LaTeX and step-by-step reasoning.



The interface makes PrismMathQA inspectable as an agentic tutoring system. The profile fields show what learner context is available before generation, the question area shows the exact problem submitted to the tutor, and the log path makes the run auditable after the session. The answer panel demonstrates the expected learner experience: the system does not only output (a, b, c), but explains how to derive $\sin t$ and $\cos t$, substitute them into $\sin^2 t + \cos^2 t = 1$, and then specialize the result

to $X = 64$. This connects the UI to the framework design: retrieval and memory provide context, the reasoning model produces the explanation, and symbolic verification can be used when exact computation matters.



Model Answer

Great question, An! Understanding how to turn a parametric description into an implicit equation (like the $ax^2 + bxy + cy^2 = 1$ form you just worked with) draws on several interconnected ideas. Here's a roadmap of the topics you'll want to review, organized in the order that usually helps the most:

1. Parametric Curves

What to know	Why it helps
How to write $x(t)$ **and** $y(t)$ and interpret the parameter t (often an angle)	Lets you see the geometric meaning of the curve (e.g., circles, ellipses, Lissajous figures).
Eliminating the parameter (solving one equation for $\sin t$ or $\cos t$ and substituting)	Gives the implicit equation you need for the "$ax^2 + bxy + cy^2 = 1$" form.
Common tricks – using $\sin^2 t + \cos^2 t = 1$, double-angle formulas, sum-to-product, etc.	Provides the algebraic bridge from trigonometric functions to polynomial relations.

Suggested practice: Take simple parametrizations like $(x, y) = (\cos t, \sin t)$ (unit circle) or $(x, y) = (a \cos t, b \sin t)$ (ellipse) and derive their implicit equations.

2. Trigonometric Identities

- **Fundamental identity:** $\sin^2 t + \cos^2 t = 1$ (the work-horse for elimination).
- **Sum-to-product & product-to-sum:** Useful when you see expressions like $\cos t - \sin t$.
- **Double-angle & half-angle formulas:** Appear when the parameter is multiplied (e.g., $\cos 2t$).

Tip: Keep a small "cheat sheet" of the most common identities; you'll often recognize patterns quickly.

3. Algebraic Manipulation

- **Solving for a variable** (e.g., $\sin t = y/X$).

The second screenshot illustrates the follow-up behavior. Instead of treating the next message as unrelated, the tutor refers back to the same mathematical task: converting a parametric description into an implicit equation of the form $ax^2 + bxy + cy^2 = 1$. The response uses the learner's name and turns the prior solution into a study roadmap covering parametric curves, trigonometric identities, and algebraic manipulation. This demonstrates the main value of memory in PrismMathQA: a follow-up answer can support learning goals and topic review, not merely repeat the previous final answer.

6 Conclusion

PrismMathQA shows how an LLM-based math tutor can combine explanation with explicit agentic support. Retrieval grounds answers in MetaMathQA examples, memory preserves learner context across turns, and a SymPy-backed verifier checks exact symbolic computation. Together, these components turn a stateless answer generator into a tutoring framework that can ground, adapt, and verify mathematical reasoning.

The report also positions AVSD as the finetuning method for the reasoning model. AVSD uses full solutions, partial rationales, final answers, and concise hints as privileged views, then balances cross-view consensus with gated residual support. The remaining work is to evaluate a trained PrismMathQA checkpoint end to end: build a unified Avg@4 harness, improve retrieval for new problem turns, add learner summaries and misconception tracking, expand the math tool set, and connect AVSD training to the deployed tutoring loop.

References

Rishabh Agarwal, Nino Vieillard, Yongchao Zhou, Piotr Stanczyk, Sabela Ramos Garea, Matthieu Geist, and Olivier Bachem. On-policy distillation of language models: Learning from self-

generated mistakes. In *International Conference on Learning Representations*, 2024. URL https://openreview.net/forum?id=3zKtaqxLhW.

Mislav Balunovic, Jasper Dekoninck, Ivo Petrov, Nikola Jovanovic, and Martin Vechev. Matharena: Evaluating LLMs on uncontaminated math competitions, 2025. URL https://matharena.ai/.

Karl Cobbe, Vineet Kosaraju, Mohammad Bavarian, Mark Chen, Heewoo Jun, Lukasz Kaiser, Matthias Plappert, Jerry Tworek, Jacob Hilton, Reiichiro Nakano, Christopher Hesse, and John Schulman. Training verifiers to solve math word problems. *arXiv preprint arXiv:2110.14168*, 2021.

DeepSeek-AI. DeepSeek-R1: Incentivizing reasoning capability in LLMs via reinforcement learning. *arXiv preprint arXiv:2501.12948*, 2025.

Geoffrey Hinton, Oriol Vinyals, and Jeff Dean. Distilling the knowledge in a neural network. *arXiv preprint arXiv:1503.02531*, 2015.

Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich Kuttler, Mike Lewis, Wen-tau Yih, Tim Rocktäschel, Sebastian Riedel, and Douwe Kiela. Retrieval-augmented generation for knowledge-intensive NLP tasks. *Advances in Neural Information Processing Systems*, 33:9459–9474, 2020.

Aaron Meurer, Christopher P. Smith, Mateusz Paprocki, Ondrej Certik, Sergey B. Kirpichev, Matthew Rocklin, Amit Kumar, Sergiu Ivanov, Jason K. Moore, Sartaj Singh, Thilina Rathnayake, Sean Vig, Brian E. Granger, Richard P. Muller, Francesco Bonazzi, Harsh Gupta, Shivam Vats, Fredrik Johansson, Fabian Pedregosa, Matthew J. Curry, Andy R. Terrel, Stepan Roucka, Ashutosh Saboo, Isuru Fernando, Sumith Kulal, Robert Cimrman, and Anthony Scopatz. SymPy: Symbolic computing in python. *PeerJ Computer Science*, 3:e103, 2017.

Duy Nguyen, Hanqi Xiao, Archiki Prasad, Zaid Khan, Anirban Das, Austin Zhang, Sambit Sahu, Hyunji Lee, Elias Stengel-Eskin, and Mohit Bansal. AVSD: Adaptive-view self-distillation by balancing consensus and teacher-specific privileged signals. *arXiv preprint arXiv:2605.20643*, 2026.

Zhihong Shao, Peiyi Wang, Qihao Zhu, Runxin Xu, Junxiao Song, Xiao Bi, Haowei Zhang, Mingchuan Zhang, Y. K. Li, Y. Wu, and Daya Guo. DeepSeekMath: Pushing the limits of mathematical reasoning in open language models. *arXiv preprint arXiv:2402.03300*, 2024.

Chenxu Yang, Chuanyu Qin, Qingyi Si, Minghui Chen, Naibin Gu, Dingyu Yao, Zheng Lin, Weiping Wang, Jiaqi Wang, and Nan Duan. Self-distilled RLVR. *arXiv preprint arXiv:2604.03128*, 2026. doi: 10.48550/arXiv.2604.03128. URL https://arxiv.org/abs/2604.03128.

Yifan Zhang and Team Math-AI. American invitational mathematics examination (AIME) 2024, 2024.

Yifan Zhang and Team Math-AI. American invitational mathematics examination (AIME) 2025, 2025.

Siyan Zhao, Zhihui Xie, Mengchen Liu, Jing Huang, Guan Pang, Feiyu Chen, and Aditya Grover. Self-distilled reasoner: On-policy self-distillation for large language models. *arXiv preprint arXiv:2601.18734*, 2026.